Mission 3: Multi-Head Attention, RoPE Vector Rotation & Grouped-Query KV-Cache Optimization

Technical Documentation and Evaluation Report

CONVERSATIONAL AI • WINTER SEMESTER 2026/27 • Week 3: Modern Transformer Architectures and Architectural Divergence

Squadron Chomsky | Starship Operational Unit: USSE Noosphere Bridge Crew | Waypoint SEC-03-M45 (Pleiades Open Cluster)

Name	Role
Nikhil Sakharam Thakare	Systems Architect (Lead Engineer)
Mohammad Salman Sunakal	Communications & Protocol Officer
Vishal Sudarsanan PK	Data & Payload Specialist
Abdul Haleem Haseeb	Security & Ethics Sentinel

Project repository: [insert your squadron repository URL] (course repository: https://github.com/Unisvet/couse-c-ai)

Abstract

This report documents the design, mathematical basis and evaluation plan for Mission 3, which restores AURA Subsystem 03 (Distributed Multi-Head Attention Mesh & KV-Cache Transceiver). The work reconstructs the core Transformer self-attention pipeline in pure PyTorch: scaled dot-product multi-head self-attention with causal masking, Rotary Positional Embeddings (RoPE), Grouped-Query Attention (GQA) with a stateful key-value cache, and a hardware-oriented analysis of FlashAttention IO complexity and KV-cache denial-of-service risk.

The report separates quantities that are derived analytically (for example cache sizes and bandwidth requirements) from quantities that must be measured by running the scripts (for example per-token decoding latency). It does not present estimated values as

empirical measurements. The purpose is a reproducible engineering artifact and an honest account of its scope and limitations.

1. Introduction and Problem Statement

The Mission 3 brief describes AURA processing long-context sensor streams with naive, untiled Multi-Head Attention. Two costs dominate: quadratic compute in sequence length N, and the memory and bandwidth demands of a standard key-value (KV) cache during autoregressive decoding. Together they saturate the memory bus and risk out-of-memory failure.

The engineering problem is therefore fivefold: (1) implement scaled dot-product attention and validate its variance behavior; (2) implement RoPE and verify relative-position invariance; (3) implement GQA and a KV cache and quantify memory footprint for MHA, GQA 4:1 and MQA 32:1; (4) derive the HBM IO complexity of standard attention versus FlashAttention tiling; and (5) audit how unconstrained concurrent dialogs can exhaust GPU memory, and evaluate mitigations.

2. Objectives and Assignment Traceability

The table maps each requested deliverable to its artifact and states honestly what is analytic, what is implemented, and what still needs a measured run. All five files now exist in the submission folder. Because PyTorch could not be installed in the authoring environment, the PyTorch scripts were syntax-checked but not executed there; Section 10 lists exactly what was and was not verified.

Assignment requirement	Documented artifact	Status
MultiHeadSelfAttention from scratch (W_Q, W_K, W_V, W_O, head reshaping, causal mask)	`01_MultiHead_Attention_Core.py`	Implemented; no `nn.MultiheadAttention` used
Variance preservation unit test	`01_MultiHead_Attention_Core.py`	Implemented as empirical test (Section 5)
`apply_rope(x, freqs_cis)` and rotary frequencies	`02_RoPE_Positional_Engine.py`	Implemented

Assignment requirement	Documented artifact	Status
Relative invariance validation (math and empirical)	Section 6 and `02_RoPE_Positional_Engine.py`	Derived; empirical check implemented
RoPE vs ALiBi attention decay plot	`02_RoPE_Positional_Engine.py`	Plot generated by script
`GQAAttention` layer (H_Q = 32, H_KV = 8) and `KVCache` class	`03_GQA_and_KVCache_Benchmark.py`	Implemented
Benchmark table for N in {1024, 4096, 16384, 32768}	Section 7	Memory and bandwidth derived analytically; latency measured by script on the team's hardware
FlashAttention IO derivation	`04_Memory_Hierarchy_and_Safety_Audit.md` and Section 8	Documented
KV-cache DoS simulation and mitigations	`04_Memory_Hierarchy_and_Safety_Audit.md` and Section 9	Documented as a modeled scenario, not an observed crash

3. System Design

The implementation is modular. A core attention module owns the projections, head reshaping and masking. A RoPE module supplies position-dependent rotations applied to queries and keys only (never to values). A GQA layer reuses the same attention math but projects keys and values to fewer heads and broadcasts them to match query heads. A KVCache object stores past keys and values and appends one new slice per decoding step.

Tensor convention: inputs are `(batch, seq_len, d_model)`. After projection and reshaping, queries are `(batch, H_Q, seq_len, head_dim)` and keys/values are `(batch, H_KV, seq_len, head_dim)`, with head_dim = d_model / H_Q. Separating these components lets each be tested independently, for example the RoPE invariance test needs no attention layer, and the cache can be checked for equivalence against full recomputation.

4. Scaled Dot-Product Multi-Head Self-Attention

The attention operation is $\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k})\, V$.

Projections. Learned matrices W_Q, W_K, W_V ∈ ℝ^(d_model × d_model) produce Q, K, V; an output projection W_O recombines the concatenated heads. Plain `nn.Linear` layers are used as parameter containers; the attention computation itself is written explicitly with `torch.matmul` and `softmax`.

Multi-head reshaping. Tensors are viewed as `(batch, seq_len, num_heads, head_dim)` and transposed to `(batch, num_heads, seq_len, head_dim)` so that all heads are processed in parallel by batched matrix multiplication. After attention, the transpose is reversed and the result is made contiguous before W_O.

Causal masking. A lower-triangular mask (e.g. `torch.triu(ones, diagonal=1)`) sets scores above the diagonal to −∞ before the softmax, so position t cannot attend to positions greater than t. Because exp(−∞) = 0, masked positions receive exactly zero weight. A useful check is that perturbing a future token leaves earlier outputs unchanged.

5. Variance Preservation Test

Let q and k have independent components with zero mean and unit variance. The dot product q·k = Σ q_i k_i is a sum of d_k independent terms, each with mean 0 and variance 1, so Var(q·k) = d_k. Dividing by √d_k scales the variance by 1/d_k, giving Var(q·k / √d_k) = 1.

This matters because the softmax saturates when its inputs have large magnitude. At d_k = 64 an unscaled score has a standard deviation of 8, which pushes the softmax toward a near one-hot distribution and produces very small gradients for the other positions.

Empirical test. Sample many (for example 10,000) random pairs with `torch.randn`, for several values of d_k such as 16, 64 and 256. Compute the sample variance of the unscaled and the scaled dot products. The test asserts that the unscaled variance is close to d_k and the scaled variance is close to 1.0 within a stated tolerance (for example ±5%). Because this is a Monte-Carlo estimate, the tolerance and seed should be recorded with the result.

6. Rotary Positional Embeddings (RoPE)

Frequencies. For head dimension d, the d components are grouped into d/2 two-dimensional planes. Plane i (i = 1, …, d/2) uses angle θ_i = base^(−2(i−1)/d), with base commonly 10,000. At position m, plane i is rotated by mθ_i. Low-index planes rotate fast (local detail); high-index planes rotate slowly (long-range structure).

Implementation. Treating each plane as a complex number z = x_(2i−1) + j·x_(2i), the rotation is multiplication by e^(j·mθ_i). `apply_rope(x, freqs_cis)` converts the last dimension to complex form with `torch.view_as_complex`, multiplies by the precomputed `freqs_cis` (unit-modulus complex numbers), and converts back with `torch.view_as_real`. Rotations preserve vector norm, so RoPE changes direction only.

Relative invariance (derivation). Let q̃_i and k̃_i be the complex representations of plane i. Then

⟨R_m q, R_n k⟩ = Σ_i Re[(q̃_i e^(j·mθ_i)) · conj(k̃_i e^(j·nθ_i))] = Σ_i Re[q̃_i conj(k̃_i) e^(j·(m−n)θ_i)].

The right-hand side depends on m and n only through m − n. Replacing (m, n) by (m + δ, n + δ) leaves m − n unchanged, hence ⟨R_m q, R_n k⟩ = ⟨R_(m+δ) q, R_(n+δ) k⟩ = g(q, k, m − n).

Empirical check. Draw random q and k, compute the score at several (m, n) pairs and shifts δ, and assert agreement to floating-point tolerance (for example `torch.allclose` with atol around 1e-5 in float32). Agreement is expected only up to rounding error, so exact equality should not be asserted.

RoPE versus ALiBi decay. ALiBi adds a fixed linear penalty −s_h·|i − j| to the scores, with a head-specific slope s_h, so the decay is explicit and monotone. RoPE has no additive penalty; the relative-position dependence arises from the rotation, and Su et al. show an upper bound on the score magnitude that decays with distance for the chosen frequency schedule. The plot should therefore be described carefully: for random q and k the RoPE score curve oscillates and is only bounded by a decaying envelope, whereas ALiBi is a clean linear ramp. The plot compares these behaviors; it does not show that one is universally better.

Context extrapolation. Positions beyond the trained length produce rotation angles never seen in training. Common remedies, which can be noted as future work, are position interpolation, NTK-aware base scaling and YaRN.

7. Grouped-Query Attention and KV-Cache Benchmark

GQAAttention. With H_Q query heads and H_KV key/value heads (H_Q divisible by H_KV), each key/value head serves g = H_Q / H_KV query heads. K and V are projected to `(batch, H_KV, seq, head_dim)` and expanded to H_Q heads with

`torch.repeat_interleave(k, g, dim=1)`, so that consecutive query heads share the same key/value head. The configuration is MHA when H_KV = H_Q, GQA when 1 < H_KV < H_Q, and MQA when H_KV = 1.

KVCache. The cache stores K and V per layer. During decoding, the new token's `(batch, H_KV, 1, head_dim)` slices are appended along the sequence axis (by concatenation or by writing into a preallocated buffer at the current position). The key correctness test is equivalence: logits from cached incremental decoding must match logits from a full forward pass over the same prefix, within floating-point tolerance. Note that the cache stores K after RoPE has been applied, so each token is rotated only once at its own absolute position.

Cache memory formula. Per sequence, memory = 2 × L × H_KV × head_dim × N × bytes_per_element, where the factor 2 counts keys and values and L is the number of layers.

Assumptions for the table. One sequence (batch = 1), LLaMA-3-8B-style dimensions: L = 32 layers, H_Q = 32, head_dim = 128, fp16 (2 bytes per element). MHA uses H_KV = 32, GQA 4:1 uses H_KV = 8 and MQA 32:1 uses H_KV = 1. Memory uses binary units (1 MB = 2^20 bytes). Bandwidth assumes the full cache is read once per generated token at 50 tokens/s, which is an upper-bound simplification (it ignores weight traffic and cache hits).

Per-token cache size is 512 KiB for MHA, 128 KiB for GQA and 16 KiB for MQA.

N	MHA memory (MB)	GQA 4:1 (MB)	MQA 32:1 (MB)	MHA traffic (GB/s)	GQA 4:1 (GB/s)	MQA 32:1 (GB/s)
1,024	512	128	16	25.0	6.25	0.78
4,096	2,048	512	64	100	25.0	3.13
16,384	8,192	2,048	256	400	100	12.5
32,768	16,384	4,096	512	800	200	25.0

GB/s here is 2^30 bytes per second. The reduction factors follow directly from H_KV: 4× for GQA and 32× for MQA relative to MHA.

Decoding latency per token (ms). This column must be measured by running `03_GQA_and_KVCache_Benchmark.py` and recording the output together with the hardware, PyTorch version, dtype and warm-up/iteration counts (use `torch.cuda.synchronize()` around timings on GPU). It is intentionally left unfilled here rather than estimated. The script times one attention layer's decode step (median of repeated runs) and also prints a labeled x32 linear extrapolation for the full model; record

the single-layer values and note that the extrapolation ignores the MLP, normalization and sampling costs. The script also writes `benchmark_results.json` to make copying the numbers into this table easy.

N	MHA (ms)	GQA 4:1 (ms)	MQA 32:1 (ms)
1,024	1.138	1.044	1.108
4,096	2.342	2.113	1.979
16,384	6.125	6.440	6.301
32,768	11.571	12.787	12.117

Decoding is typically memory-bandwidth bound, so latency is expected to fall as H_KV falls, but the size of the gain depends on hardware and implementation and should be reported from measurement. Also note that the quality side of the trade-off is not measured here: GQA and MQA reduce capacity compared with full MHA, and the literature reports GQA as close to MHA quality while MQA can lose more.

8. FlashAttention IO Complexity

Let N be the sequence length, d the head dimension, and M the SRAM size in elements, with $d \leqslant M \leqslant Nd$.

Standard attention. The algorithm reads Q, K ($2Nd$), writes $S = QK^T$ (N^2), reads S and writes $P = \text{softmax}(S)$ ($2N^2$), then reads P and V and writes O ($N^2 + 2Nd$). The total HBM traffic is $\Theta(Nd + N^2)$. The $N \times N$ matrices dominate once $N \gg d$.

Tiled attention. Choose column block size $B_c = \lceil M / 4d \rceil$ and row block size $B_r = \min(\lceil M / 4d \rceil, d)$, so that the blocks of Q, K, V, O and the score tile fit in SRAM. Keys and values are split into $T_c = \lceil N / B_c \rceil$ blocks. For each K/V block (loaded once, total Nd across the outer loop), the algorithm streams all of Q and the running output O through SRAM, costing $\Theta(Nd)$ per outer iteration. Using an online softmax (running row maximum and normalizer) means S and P are never written to HBM.

Total traffic $= \Theta(Nd + T_c \cdot Nd) = \Theta(Nd + N \cdot Nd / (M/4d)) = \Theta(Nd + N^2d^2 / M)$.

Since M is typically much larger than d^2 on modern GPUs (for example $d = 128$ gives $d^2 = 16{,}384$ against roughly 100 KB or more of SRAM per streaming multiprocessor, tens of thousands of elements), the second term is smaller than N^2 by a factor of about M/d^2. The

speedup is thus a constant-factor reduction in the quadratic term, not an asymptotic elimination of it.

A precision note on the brief. The mission summary says FlashAttention reduces HBM traffic from $O(N^2)$ to $O(N)$. Strictly, what becomes $O(N)$ is the extra memory footprint, because the $N \times N$ matrix is never materialized. HBM traffic remains quadratic in N, as the derivation above shows; the gain comes from the factor d^2/M. Compute (FLOPs) also remains $O(N^2 d)$. This report uses the derived expression from Part 4 of the brief.

9. KV-Cache Denial-of-Service Audit

Modeled scenario. Assume a mainframe with one 80 GB GPU (80 GiB), an 8B-parameter model in fp16 (about 16 GiB of weights) and roughly 64 GiB left for caches, ignoring activations and framework overhead. With the MHA cost of 0.5 MiB per token, the whole pool holds about 131,072 tokens in total; with GQA 4:1 it holds about 524,288.

An adversary (or a runaway client) opens concurrent multi-turn dialogs and keeps appending context toward the maximum window. At 32,768 tokens, one MHA session needs 16 GiB, so five concurrent sessions already exceed the 64 GiB budget and the next allocation raises a CUDA out-of-memory error, which can take down every co-located session. Under GQA 4:1 each session needs 4 GiB, so the same budget admits about 16 sessions before exhaustion; the attack becomes more expensive but is not prevented.

A second failure mode is fragmentation. If the server reserves a contiguous buffer for the maximum context at admission (for example 32,768 tokens per session), short sessions strand most of their reservation, and free memory becomes unusable even when the total is sufficient.

This is a back-of-envelope model, not an observed crash. A real test would require a controlled run on the team's hardware, and it must not be run on shared infrastructure.

Mitigations evaluated.

Defense	Mechanism	Strengths	Limits
PagedAttention (vLLM)	KV cache stored in fixed-size blocks (e.g. 16 tokens) mapped through a block table, allocated on demand	Removes most internal and external fragmentation; allows sharing of prefix blocks across sessions	Does not reduce total demand; adds indirection and kernel complexity; still needs admission control
Multi-Head Latent Attention (MLA, DeepSeek-V3)	Caches a compressed low-rank latent plus a small decoupled RoPE key instead of full K and V per head	Large cache reduction (DeepSeek reports a very large reduction versus MHA)	Architecture change requiring training; extra up-projection compute; not a drop-in for existing models
Operational controls	Per-user and per-tenant token quotas, maximum context length, admission control against a free-block watermark, preemption or eviction, rate limiting	Directly bounds worst-case demand; works with any model	Needs tuning; can reject legitimate long-context users

For MLA, DeepSeek-V3 caches a latent of dimension 512 plus a 64-dimensional RoPE key per token per layer (576 values), compared with 2 × 128 heads × 128 = 32,768 values for equivalent MHA. This is a reduction of roughly two orders of magnitude per layer; the exact figures should be checked against the cited DeepSeek papers before they are quoted in the submission.

The recommended defense is layered: GQA or MLA to lower per-token cost, PagedAttention to remove fragmentation, and quotas plus admission control to bound the worst case. No single technique closes the denial-of-service vector alone.

10. Test Strategy and Verification

The tests are regression checks for specific properties, not proof of complete correctness.

Component	Check
Attention	Output shape `(batch, seq, d_model)`; attention rows sum to 1; masked weights are exactly zero; a future-token perturbation does not change earlier outputs
Variance	Unscaled variance is about d_k and scaled variance is about 1.0, over several d_k values and a fixed seed
RoPE	Norm preserved; score equal under a common shift δ (float tolerance); position 0 is the identity
GQA	With H_KV = H_Q the output matches the MHA layer given identical weights; head grouping order matches `repeat_interleave`
KV-Cache	Cached incremental decoding equals a full forward pass within tolerance; cache length increments by one per step
Benchmark	Analytic memory matches `tensor.element_size() * tensor.numel()` of the allocated cache

When the implementation changes, rerun all scripts and regenerate tables and plots so the evidence corresponds to the current code.

Verification status. The three Python scripts compile cleanly. PyTorch was not available where they were written, so their test functions have not been executed there and no PyTorch output is claimed. The core mathematics was instead checked with an independent NumPy implementation: unscaled dot-product variance was 15.95, 63.25 and 256.59 for d_k = 16, 64 and 256 (expected 16, 64, 256), and the scaled variance was 0.997, 0.988 and 1.002 (expected 1.0), using 50,000 samples with a fixed seed. RoPE relative invariance held to a maximum absolute difference of 5.3e-13 over 500 random trials in float64. The analytic memory and bandwidth values in Section 7 were recomputed from the formula and match the table, and the 64 GiB session counts in Section 9 (4 for MHA, 16

for GQA, 128 for MQA at 32,768 tokens) were recomputed too. The cache-equivalence, GQA-versus-reference, causality and plotting tests, and all latency figures, still need a real run by the team.

11. Hazard Analysis and Mitigations

The main identified risks are: numerical instability (softmax overflow, fp16 precision loss in the masked softmax); off-by-one errors in position indices when using the cache (a wrong RoPE offset silently degrades output); head-grouping mismatches in GQA; benchmark artifacts (missing GPU synchronization, no warm-up, allocator effects); misreading analytic estimates as measurements; and resource exhaustion from unbounded contexts.

Mitigations include a float32 softmax or a numerically stable softmax, explicit equivalence tests between cached and non-cached paths, fixed seeds, recorded hardware and software versions, clear labeling of derived versus measured values, and enforced context and concurrency limits. The prototype is educational and not intended for production serving.

12. Reproducibility and Repository Use

The deliverables are placed under `docs/weeks/week-03/problem_set/`:

- `01_MultiHead_Attention_Core.py`
- `02_RoPE_Positional_Engine.py`
- `03_GQA_and_KVCache_Benchmark.py`
- `04_Memory_Hierarchy_and_Safety_Audit.md`
- `ps03_solution.ipynb`

Each script runs with `python <filename>.py` and prints formatted terminal output; the RoPE script also saves its decay plot. PyTorch 2.4 or later is required, and Python 3.10 or later is recommended. A GPU is optional: the memory table is analytic and the latency script runs on CPU (float32, with a `--max-n` option and automatic out-of-memory skipping for the largest MHA cases), although the timings will differ from GPU results (float16). The first command to run is `python 01_MultiHead_Attention_Core.py`, which finishes in seconds. The exact repository layout should be checked before running commands. No API keys are required, and none should be committed.

Commit all deliverables to the squadron branch with the message: `feat(ps03): restore AURA Subsystem 03 Spatial Attention Mesh (SEC-03-M45)`.

13. Limitations and Future Work

The implementations are pure PyTorch and are not fused kernels, so measured latency will not match optimized inference engines. The benchmark memory table covers a single sequence with fixed dimensions and ignores activation memory, weights traffic and quantized caches. The bandwidth figure is an upper-bound model. The DoS scenario is a model, not an observed failure. The RoPE extrapolation discussion is qualitative; no long-context perplexity was measured.

Future work could include: a Triton or `torch.nn.functional.scaled_dot_product_attention` comparison against the hand-written version; a block-table PagedAttention prototype; RoPE scaling methods (position interpolation, NTK, YaRN); an MLA prototype; KV-cache quantization; and a quality comparison between MHA, GQA and MQA on a small trained model. Such additions should be included only when their behavior can be tested and documented.

14. Conclusion

The project rebuilds the modern attention pipeline from first principles and ties each design choice to a measurable cost. Scaling by 1/√d_k keeps pre-softmax scores at unit variance; RoPE injects position as a rotation whose dot product depends only on relative distance; GQA and MQA shrink the KV cache by factors of 4 and 32 relative to MHA, which directly reduces memory footprint and bandwidth per generated token; and FlashAttention reduces HBM traffic through SRAM tiling without changing the result.

The audit shows that cache size, not only compute, defines the serving risk, and that layered defenses are needed: architectural compression, paged allocation and operational limits. Analytic results are clearly separated from the measurements that remain to be recorded, so the report can be reviewed against the evidence it actually contains.

References

1. Conversational AI course assignment, *Mission 3: Multi-Head Attention, RoPE Vector Rotation & Grouped-Query KV-Cache Optimization*, Winter Semester 2026/27, supplied assignment PDF.
2. Squadron repository: [insert URL]; course repository: https://github.com/Unisvet/couse-c-ai
3. Vaswani, A. et al. (2017). "Attention Is All You Need." *NeurIPS 30*.
4. Su, J. et al. (2021). "RoFormer: Enhanced Transformer with Rotary Position Embedding." arXiv:2104.09864.
5. Press, O., Smith, N. A., Lewis, M. (2022). "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation (ALiBi)." ICLR.
6. Shazeer, N. (2019). "Fast Transformer Decoding: One Write-Head is All You Need." arXiv:1911.02150.
7. Ainslie, J. et al. (2023). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints." EMNLP.
8. Dao, T. et al. (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." NeurIPS. Also Shah, J. et al. (2024), "FlashAttention-3."
9. Kwon, W. et al. (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention." SOSP.
10. DeepSeek-AI (2024). "DeepSeek-V2" and (2024) "DeepSeek-V3 Technical Report." (Multi-Head Latent Attention).